

ΔΗΜΟΚΡΙΤΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΡΑΚΗΣ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ



ΔΗΜΟΚΡΙΤΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΡΑΚΗΣ  **DEMOCRITUS UNIVERSITY OF THRACE**

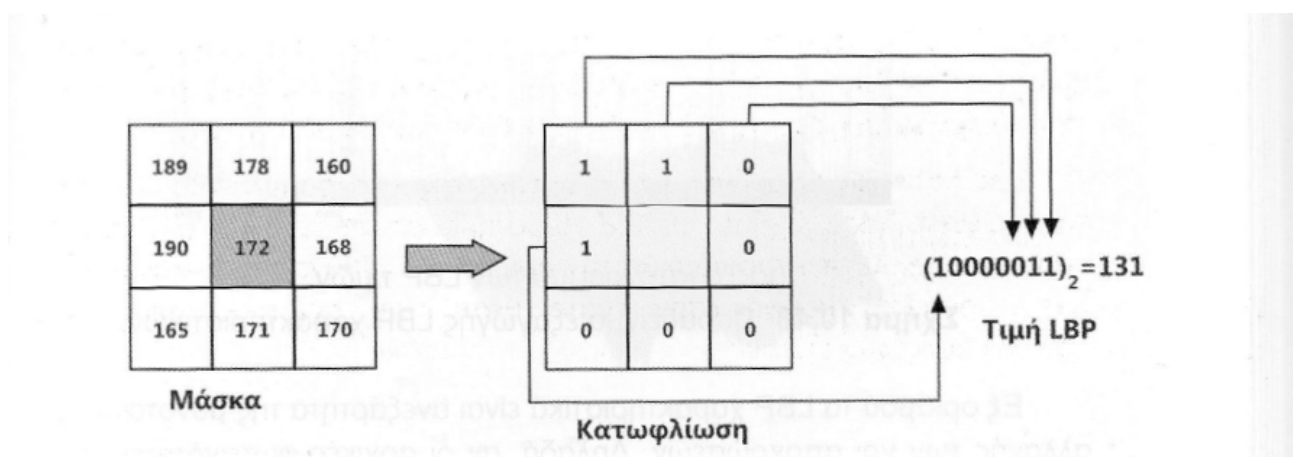
Τεχνολογία Παράλληλης Επεξεργασίας
Εργασία: Υπολογισμός Local Binary Patterns σε gray εικόνες με cuda

Χάρης Γκίζας
Θεόδωρος-Γεώργιος Καραγκιοζίδης
Σωκράτης Μπίτζης

Η εργασία μας ασχολείται με τα Local Binary Patterns, τα οποία είναι μια μέθοδος περιγραφή της υφής μιας εικόνας. Ο σκοπός της εργασίας μας είναι να πάρουμε μια γκρίζα εικόνα και να βρούμε τις τιμές LBP της και να τις κάνουμε εικόνα, με κώδικα γραμμένο σε cuda.

Το LBP είναι ένα πολύ γνωστό και ευρέως χρησιμοποιούμενο χαρακτηριστικό για πολλές εφαρμογές εικόνας, όπως η αναγνώριση προσώπου, η αναγνώριση δακτυλικών αποτυπωμάτων και άλλα προβλήματα ταξινόμησης.

Για να υπολογίσουμε τις τιμές LBP διατρέχουμε ένα ένα τα εικονοστοιχεία μίας εικόνας 8-bit grayscale που στην ουσία είναι ένας πίνακας 2 διαστάσεων με τιμές φωτεινότητας από 0(μαύρο) μέχρι 255(άσπρο) και για κάθε ένα από αυτά δημιουργούμε μια προσωρινή μεταβλητή αρχικοποιημένη στο 0 στην οποία προσθέτουμε τις τιμές 128,64,...,1 ανάλογα με το αν το αντίστοιχο γειτονικό εικονοστοιχείο έχει φωτεινότητα μεγαλύτερη του κεντρικού, αλλιώς 0.



Ακολουθεί η Cuda συνάρτηση υπεύθυνη για αυτήν την διαδικασία:

```
__global__ void CalcLBP(unsigned char *Inimg, unsigned char *Outimg) {
    int id = threadIdx.x + blockIdx.x * blockDim.x;
    if (id < size) {
        unsigned char T = Inimg[id]; // targeted pixel
        unsigned char bt = 0; // temporary lbp value
        if ( (id % w) > 0 && Inimg[id-1] > T ) bt += 128;
        if ( (id % w) > 0 && id < (w*(h-1)) && Inimg[id+w-1] > T ) bt += 64;
        if ( id < (w*(h-1)) && Inimg[id+w] > T ) bt += 32;
        if ( (id % w) < (w-1) && id < (w*(h-1)) && Inimg[id+w+1] > T ) bt += 16;
        if ( (id % w) < (w-1) && Inimg[id+1] > T ) bt += 8;
        if ( (id % w) < (w-1) && id >= w && Inimg[id-w+1] > T ) bt += 4;
        if ( id >= w && Inimg[id-w] > T ) bt += 2;
        if ( (id % w) > 0 && id >= w && Inimg[id-w-1] > T ) bt += 1;
        Outimg[id] = bt;
    }
}
```

Έχουμε επιλέξει να διατρέξουμε τα στοιχεία της εικόνας σειριακά και όχι δισδιάστατα, χωρίζοντας την σε τμήματα(Blocks) 1024 στοιχείων(threads) όπως φαίνεται παρακάτω, και κάθε thread αντιστοιχεί σε ένα εικονοστοιχείο.

```
int blocks=size/1024+(size%1024==0?0:1);
CalcLBP<<<blocks,1024>>>(DevInimg,DevOutimg);
```

Υπάρχουν πολλές εναλλακτικές υλοποιήσεις του κώδικα όπως για παράδειγμα να χωρίσουμε την εικόνα σε δισδιάστατα blocks, να χρησιμοποιήσουμε κοινή μνήμη ή και τα δύο, αλλά στο συγκεκριμένο πρόβλημα καμία από αυτές δεν προσφέρει καλύτερα αποτελέσματα και μάλιστα η χρήση κοινής μνήμης προκαλεί μεγαλύτερα πρόβλημα μνήμης από ότι λύνει. Ο λόγος για αυτό είναι ότι οι κλήσεις μνήμης προκειμένου να αρχικοποιήσει την κοινή μνήμη είναι πολύ περισσότερες από ότι οι κλήσεις μνήμης που χρειάζεται ο απλός κώδικας.

Ένα παράδειγμα του αντίστοιχου κώδικα με την χρήση κοινής μνήμης:

```
__global__ void CalcLBP(unsigned char *Inimg,unsigned char *Outimg){
    __shared__ unsigned char Inlocal[smdim*smdim]; //local 34x34 copy
    //int bx=blockIdx.x; //block coordinates from the "global" view
    //int by=blockIdx.y;
    int tx=threadIdx.x; //thread coordinates from the local view (inside block)
    int ty=threadIdx.y;
    int row=blockIdx.y*tilledim+ty; //row number from a "global" view
    int col=blockIdx.x*tilledim+tx; //column number from a "global" view
    int id=row*w+col; //thread id from a "global" view
    int lid=ty*tilledim+tx; //local thread id inside the block
    int smid=lid+smdim+1+2*ty; //corresponding shared memory id

    if(id<size){
        if(id==0) Inlocal[0]=0; //^<
        else if(id==(w-1)) Inlocal[smdim-1]=0; //^>
        else if(id==(h-1)*w) Inlocal[(smdim-1)*smdim]=0; //v<
        else if(id==(size-1)) Inlocal[smdim*smdim-1]=0; //v>
        if(ty==0){ //up
            Inlocal[smid-smdim]=((row==0)?0:Inimg[id-w]); //^
        }
        else if(ty==(tilledim-1)){ //down
            Inlocal[smid+smdim]=((row==(h-1))?0:Inimg[id+w]); //v
        }
        if(tx==0){ //left
            Inlocal[smid-1]=((col==0)?0:Inimg[id-1]); //<
        }
        else if(tx==(tilledim-1)){ //right
            Inlocal[smid+1]=((col==(w-1))?0:Inimg[id+1]); //>
        }
        Inlocal[smid]=Inimg[id]; //do it anyway
    }
    __syncthreads();
}
```

```

if(id<size){
    unsigned char T=Inlocal[smid];
    unsigned char bt=0;
    if(Inlocal[smid-1]>T) bt+=128;
    if(Inlocal[smid+smdim-1]>T) bt+=64;
    if(Inlocal[smid+smdim]>T) bt+=32;
    if(Inlocal[smid+smdim+1]>T) bt+=16;
    if(Inlocal[smid+1]>T) bt+=8;
    if(Inlocal[smid-smdim+1]>T) bt+=4;
    if(Inlocal[smid-smdim]>T) bt+=2;
    if(Inlocal[smid-smdim-1]>T) bt+=1;
    Outimg[id]=bt;
}
__syncthreads();

```

παρατηρούμε ότι η κλήσεις των δεικτών id, smid, row, col, υπερβαίνουν κατά πολύ τον αριθμό κλήσεων του μοναδικού δείκτη id της αρχικής μορφής του κώδικα ενώ η κλήσεις του πίνακα Inimg είναι περίπου ίδιες. Επομένως καταλήγουμε να έχουμε μεγαλύτερη καθυστέρηση λόγω αυτού.

Ακολουθούν οι μετρικές του κώδικα Cuda:

	cuda	openmp64	serial	cuda speedup	openmp64 speedup	efficiency
<u>dimentions</u>	<u>ms</u>					
384x256	0,04	6,5	3,3	82,5	0,50	0,11
512x512	0,06	6,3	6,2	103,33	0,98	0,13
2550x1690	0,67	11	90	134,33	8,28	0,17
4096x4096	2,41	19	370	153,53	19,56	0,2
8192x8192	9,6	41	1019	106,15	24,52	0,14
16384x16384	38,36	113	3310	86,29	29,29	0,11

Έχουμε 4 παρατηρήσεις να κάνουμε:

-Ακόμα και για μικρές εικόνες έχουμε σημαντική επιτάχυνση χωρίς επιβάρυνση αντιθέτως με την `openmp`.

-Η επιτάχυνση είναι σημαντικά μεγαλύτερη από την `openmp` ακόμα και για 64 πυρήνες μεταξύ 80 και 150 περίπου.

-Η υψηλότερη επιτάχυνση παρατηρείται για εικόνα μεγέθους 4096x4096 πριν και μετά από την οποία η επιτάχυνση είναι μικρότερη, γεγονός που επιβεβαιώνεται κάπως από τα χαρακτηριστικά της κάρτας γραφικών που χρησιμοποιήσαμε.

Συγκεκριμένα, σύμφωνα με τα χαρακτηριστικά της, είναι αποτελεσματική για εικόνες **μέχρι και** περίπου 2500x1500 που πλησιάζει την ανάλυση της εικόνας 2550x1690 που είχαμε στην διάθεσή μας, στην οποία παρουσιάστηκε η δεύτερη μεγαλύτερη επιτάχυνση.

-Μετρώντας συμβατικά την απόδοση μένει περίπου ανάμεσα στο 0,1 και 0,2 απογοητευτικοί αριθμοί υπό άλλες συνθήκες, σύμφωνα με την φιλοσοφία της Cuda είναι φυσιολογικό.

Συμπερασματικά, η χρήση Cuda για επεξεργασία LBP είναι άκρως αποτελεσματική και ωφέλιμη.

Βιβλιογραφία:

1. Οδηγός Προγραμματισμού με τη γλώσσα C Αλέξανδρος Σ. Καράκος
2. Ψηφιακή Επεξεργασία & Ανάλυση Εικόνας Νικόλαος Η. Παπαμάρκος
3. <https://www.geeksforgeeks.org/dynamically-allocate-2d-array-c/>
4. Κώδικας και διαφάνειες παρεχόμενα από τον καθηγητή Ηρακλή Σπηλιώτη